



CS117

TRỪU TƯỢNG HÓA (ABSTRACTION) VÀ TỔNG QUÁT HÓA (GENERALIZATION)

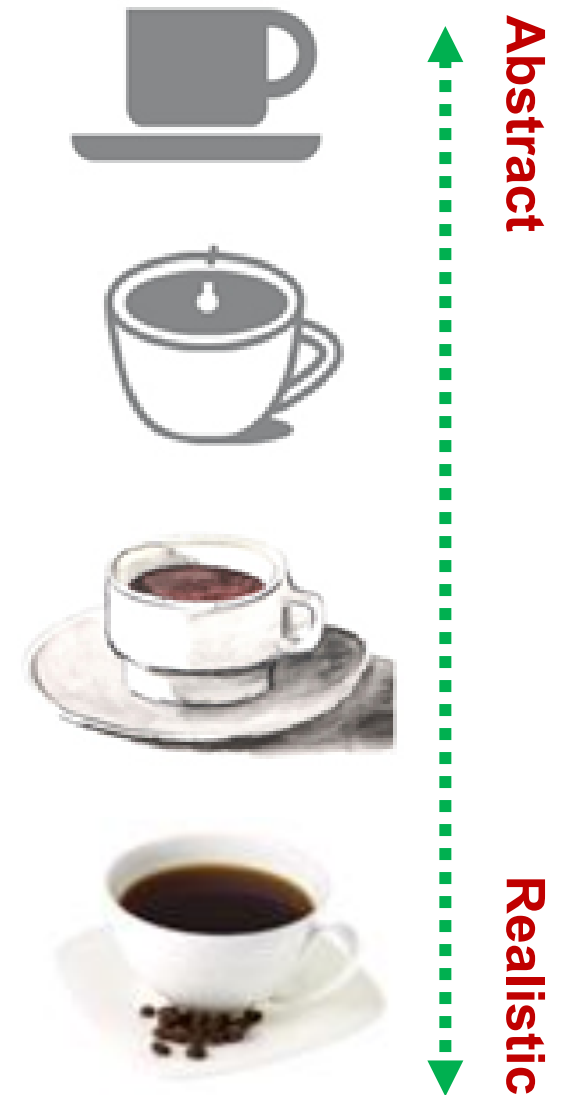
Giảng viên: NGÔ ĐỨC THÀNH



TRỪU TƯỢNG HÓA - ABSTRACTION

Trừu tượng hóa - Abstraction

- **Trừu tượng hóa (abstraction)** là quá trình loại bỏ có chọn lọc các chi tiết không liên quan để tập trung vào những thông tin cốt lõi cần thiết cho việc giải quyết vấn đề.
- Trong giải quyết bài toán,
 - Trừu tượng hóa giúp đơn giản hóa bài toán phức tạp, tập trung vào yếu tố cốt lõi.
 - Giúp tránh “sa lầy” vào chi tiết không ảnh hưởng đến mục tiêu chính. Ví dụ: giải bài toán “quản lý lớp học” → chỉ giữ thông tin: danh sách, điểm danh, lịch học.







Một số loại hình Trừu tượng hóa (Abstraction)

- **Trừu tượng hóa dữ liệu:** loại bỏ thông tin phụ, chỉ giữ giá trị cần thiết
→ Ví dụ: chỉ dùng tên và điểm, bỏ giới tính, địa chỉ trong quản lý lớp học.
- **Trừu tượng hóa hành vi:** mô tả hành động bằng trạng thái
→ Ví dụ: “đã đăng nhập”, “đang xử lý”, “đã hoàn tất”.
- **Trừu tượng hóa thuật toán:** biểu diễn ý tưởng thay vì chi tiết cài đặt
→ Ví dụ: “sắp xếp tăng dần” mà chưa cần biết dùng Bubble Sort hay Quick Sort.
- **Trừu tượng hóa kiến trúc hệ thống:** mô tả thành phần chính, mối quan hệ giữa các module.
- **Trừu tượng hóa mục tiêu:** diễn đạt bài toán ở mức khái quát
→ Ví dụ: “cần xác thực người dùng” thay vì “so khớp mã OTP và mã hệ thống”.



Một số loại hình Trừu tượng hóa (Abstraction)

- **Trừu tượng hóa trên bài toán**

- loại bỏ chi tiết không liên quan để làm rõ yêu cầu cốt lõi
- giúp hiểu rõ vấn đề cần giải quyết, tránh bị nhiễu
- Ví dụ: bài toán “quản lý lớp học” → chỉ giữ danh sách SV, điểm danh, thời khóa biểu

- **Trừu tượng hóa trên giải pháp**

- Che giấu chi tiết triển khai, chỉ giữ phần cần thiết để sử dụng hoặc mô tả
- Giúp đơn giản hóa thiết kế, tăng tính mô-đun và tái sử dụng
- Ví dụ: dùng hàm `sort(arr)` mà không cần biết thuật toán bên trong



Ví dụ, trừu tượng hóa trong lập trình & phần mềm

- Hàm (function) ẩn chi tiết xử lý, chỉ thể hiện đầu vào và đầu ra → ví dụ: `max(a, b)` không cần biết so sánh thực hiện thế nào.
- Lớp (class) trong Lập trình hướng đối tượng trừu tượng hóa đối tượng thật → ví dụ: class Student gồm tên, điểm, phương thức `calculateGPA()`.
- Interface/API là abstraction của chức năng hệ thống → người dùng chỉ gọi, không cần biết triển khai bên trong.
- Module hóa giúp chia phần mềm thành các khối chức năng rõ ràng, dễ bảo trì.
- Thiết kế phần mềm dựa trên abstraction để giảm độ phức tạp, tăng khả năng tái sử dụng.

■ *Abstraction là nguyên tắc cốt lõi trong phát triển phần mềm, giúp xây dựng hệ thống rõ ràng, hiệu quả và dễ mở rộng*

Trừu tượng hóa - Abstraction

- **Ưu điểm:**

- Giảm độ phức tạp, dễ tập trung vào phần cốt lõi của bài toán.
- Tăng khả năng tái sử dụng giải pháp ở nhiều bối cảnh khác nhau.
- Hỗ trợ thiết kế hệ thống rõ ràng, dễ bảo trì.
- Làm nền cho phát hiện mẫu (pattern recognition) và tổng quát hóa (generalization).

- **Lưu ý:**

- Nếu abstraction sai, có thể bỏ sót thông tin quan trọng → ví dụ: bỏ qua thời gian trong bài toán điểm danh → không phân biệt đi trễ.
- Abstraction quá mức dẫn đến hiểu sai bản chất hoặc đánh giá sai hiệu quả giải pháp.

■ *Cần cân bằng giữa đơn giản hóa và giữ đủ thông tin để giải quyết bài toán hiệu quả*

Trừu tượng hóa - Abstraction

1. Để đi từ ga A đến ga B thì cần đi những tuyến nào và chuyển tuyến ở đâu ?
2. Để đi từ ga A đến ga B thì cần đi qua bao nhiêu ga trung gian ?
3. Để đi từ A đến ga B mất bao nhiêu thời gian ?
4. Giá vé từ ga A đến ga B là bao nhiêu ?



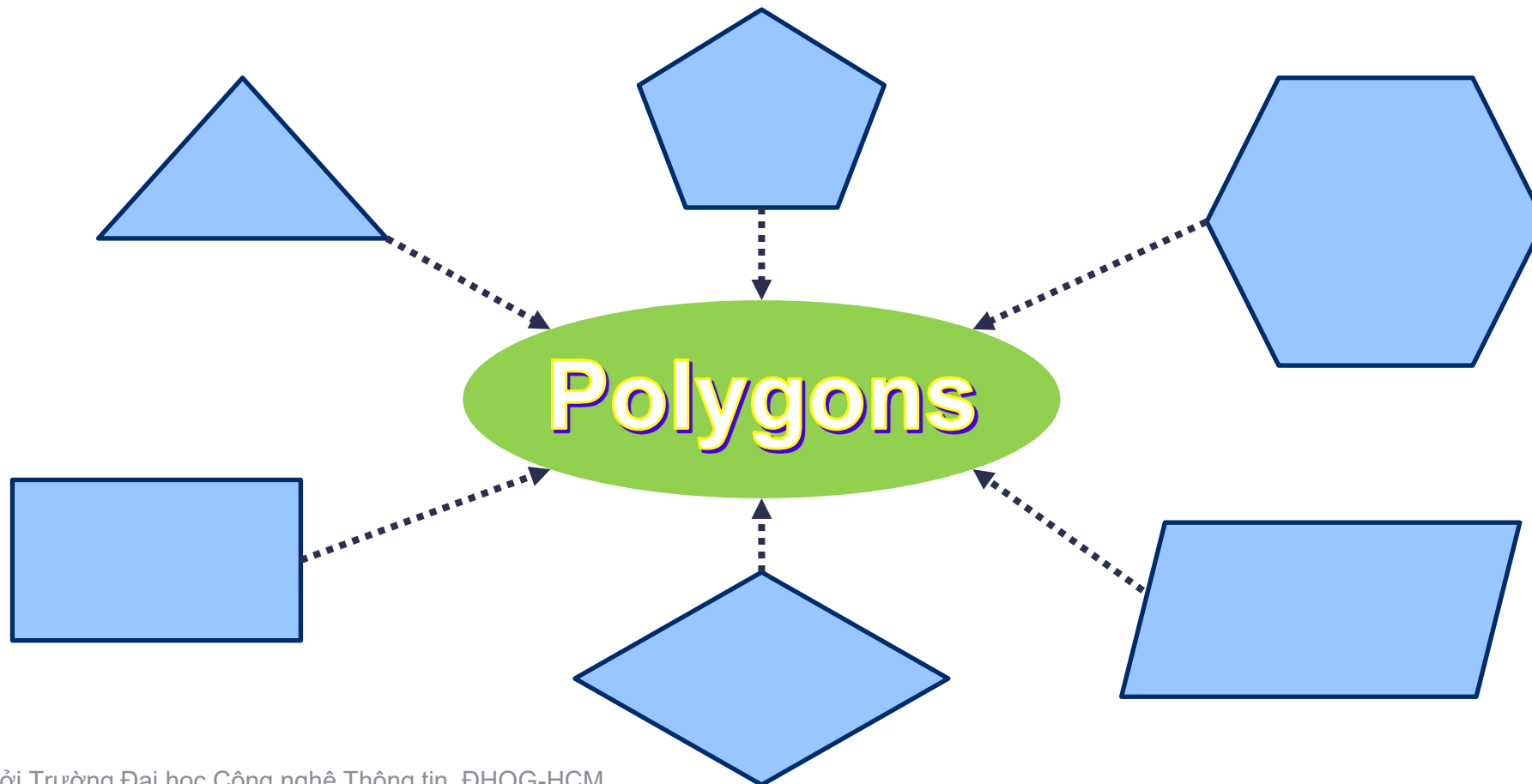


TỔNG QUÁT HÓA – GENERALIZATION



Tổng quát hóa - Generalization

■ **Tổng quát hóa (generalization)** là quá trình rút ra quy luật hoặc dạng tổng quát từ một số trường hợp cụ thể, nhằm giải quyết cả những tình huống chưa từng gặp.





Tổng quát hóa - Generalization

- **Generalization trên bài toán**

→ Tìm điểm chung giữa nhiều bài toán cụ thể → giúp phân loại và xác định bản chất bài toán → ví dụ: bài toán xếp hàng ở ngân hàng, bệnh viện... → bài toán điều phối hàng đợi → *hiểu được loại bài toán đang xử lý, từ đó dễ chọn chiến lược giải pháp phù hợp.*

- **Generalization trên giải pháp**

→ Rút ra cách giải tổng quát có thể áp dụng lại → giúp thiết kế giải pháp linh hoạt, tái sử dụng → Ví dụ: một thuật toán nhận diện khuôn mặt dùng cho nhiều ứng dụng.

- Cả hai đều cần thiết trong tư duy giải quyết vấn đề → ***một bài toán tổng quát cần lời giải tổng quát để thích ứng trong thực tế đa dạng.***



Tổng quát hóa - Generalization

Tổng quát hóa (generalization) từ nhiều trường hợp cụ thể đã biết

- ✓ Bước đầu tiên là phân tích các bài toán cụ thể có điểm tương đồng về mục tiêu, cấu trúc dữ liệu hoặc phương pháp giải.
- ✓ Xác định các yếu tố cốt lõi có thể được mô hình hóa chung.
- ✓ Hình thành chiến lược giải tổng quát có khả năng áp dụng cho nhiều bài toán cùng loại.
- ✓ Diễn đạt lời giải dưới dạng thuật toán, công thức hoặc khung giải pháp giúp tăng tính khái quát và khả năng tái sử dụng.
- ✓ Generalization đóng vai trò trung tâm trong phát triển các thư viện hàm, mô hình AI và chiến lược giải quyết bài toán ở quy mô lớn.



Tổng quát hóa khởi phát từ một bài toán cụ thể

Tổng quát hóa từ việc phân tích sâu một bài toán đơn lẻ

- ✓ Tách biệt các thành phần cố định (logic, cấu trúc) và các thành phần biến đổi (dữ liệu đầu vào, tham số), người giải có thể chuyển đổi bài toán thành một mô hình tổng quát hơn.
- ✓ Việc đặt câu hỏi "*Điều gì sẽ xảy ra nếu điều kiện thay đổi?*" giúp phát hiện phạm vi áp dụng rộng hơn của giải pháp.
- ✓ Quy trình này còn được gọi là phản tư hóa, trong đó người giải chủ động suy luận tính tổng quát từ trải nghiệm giải bài toán ban đầu.
- ✓ Biến một trường hợp cá biệt thành một nguyên mẫu (prototype) cho nhóm bài toán tiềm năng trong tương lai.



Tổng quát hóa - Generalization

- **Overgeneralization:** mở rộng quá mức dẫn đến áp dụng sai bối cảnh.
- **Generalization từ dữ liệu không đại diện** → thiếu tính khách quan. → dễ gây thiên vị (bias) và mất công bằng.
- **Không xác định rõ ràng phạm vi áp dụng:** giải pháp tổng quát nhưng không có giới hạn cụ thể → khó kiểm soát rủi ro.
- **Hiểu nhầm sự giống nhau trong mô tả là giống nhau về bản chất** → dẫn đến khái quát hóa từ các trường hợp không đủ liên quan.
- **Thiếu kiểm thử trên tình huống mới:** đánh giá generalization chỉ dựa trên dữ liệu cũ.

■ *Generalization cần được kiểm soát bằng phân tích logic, phản hồi thực tế, và kiểm tra chéo*



Kiểm tra và đánh giá khả năng Tổng quát hóa

- **Thay đổi đầu vào:** thử áp dụng giải pháp với các trường hợp có dữ liệu mới, chưa từng xuất hiện.
- **Đa dạng hóa ngữ cảnh:** kiểm tra giải pháp trong các môi trường hoặc điều kiện khác nhau → ví dụ: hệ thống hoạt động tốt cả ngày lẫn đêm, thành phố và nông thôn.
- **So sánh với giải pháp khác:** đánh giá tính phù hợp, độ linh hoạt và hiệu quả tương đối.
- **Theo dõi lỗi và ngoại lệ:** ghi nhận các trường hợp giải pháp không còn đúng → giới hạn phạm vi tổng quát hóa.
- **Phản hồi từ người dung / chuyên gia:** xác minh tính phù hợp khi triển khai thực tế.

■ *Generalization tốt không chỉ đúng về mặt logic, mà còn ổn định, đáng tin cậy khi áp dụng trong thực tế đa dạng*



TỔNG QUÁT HÓA VÀ TRỪU TƯỢNG HÓA



Trừu tượng hóa (Abstraction) và Tổng quát hóa (Generalization)

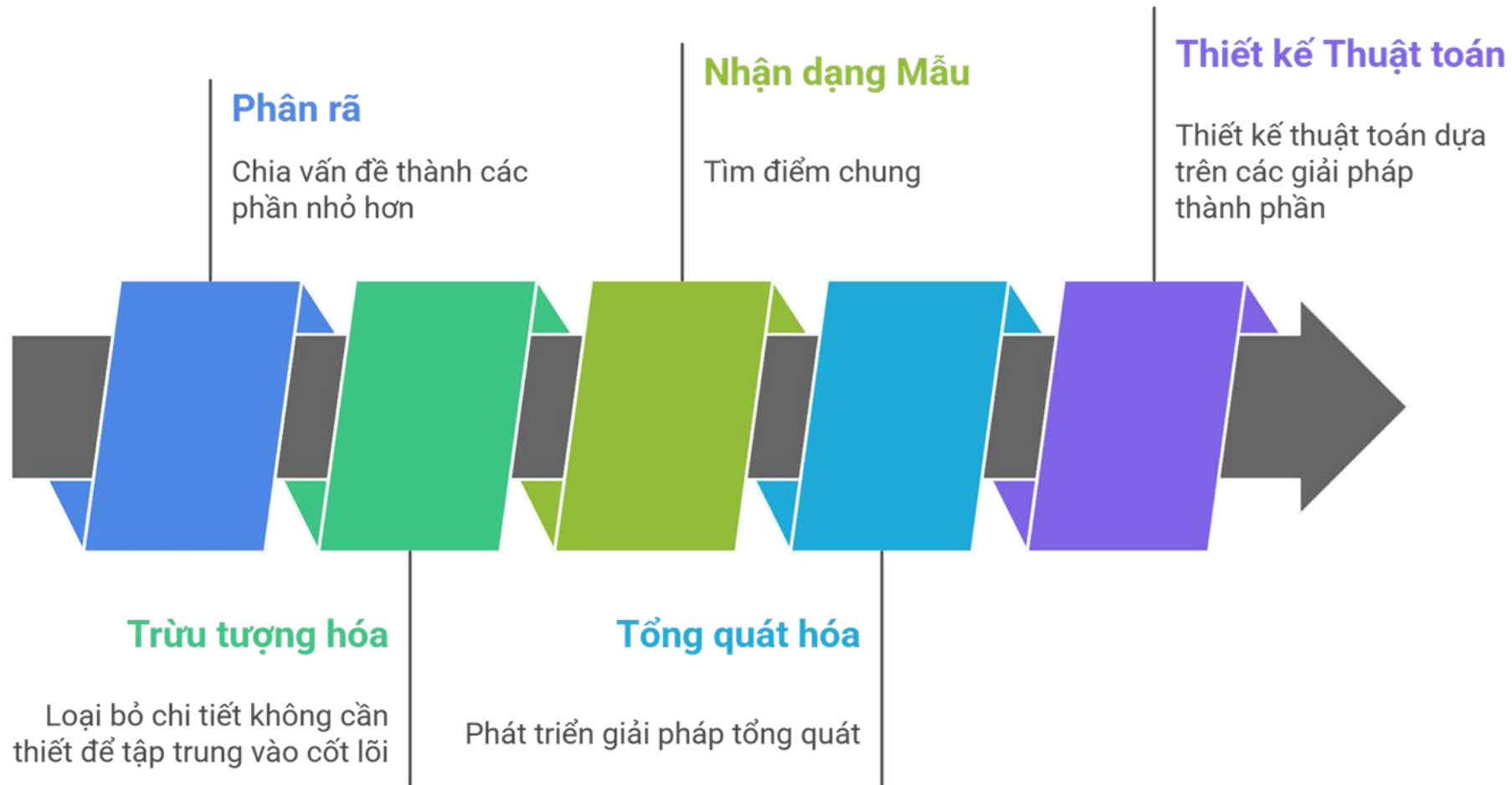
■ **Abstraction làm rõ bản chất, generalization xác định dạng bài toán và mở rộng cách giải**

- Abstraction và generalization là hai bước tư duy có liên hệ chặt chẽ trong quá trình giải quyết vấn đề.
- Abstraction tập trung vào việc đơn giản hóa bằng cách loại bỏ chi tiết không cần thiết → giúp làm rõ cấu trúc và yếu tố cốt lõi của bài toán.
- Generalization nhằm xây dựng giải pháp có thể mở rộng cho nhiều tình huống → dựa trên mẫu được phát hiện sau khi abstraction đã làm nổi bật bản chất vấn đề.
- Abstraction là điều kiện cần để nhận diện pattern và thực hiện generalization.
- Cả hai đều hỗ trợ thiết kế giải pháp có tính **hiệu quả, linh hoạt và tái sử dụng cao**.



Abstraction và Generalization trong giải quá trình giải quyết vấn đề

Trong tư duy tính toán, giải quyết vấn đề thường gồm nhiều bước nối tiếp và xen kẽ giữa: ***Decomposition, Abstraction, Generalization, Pattern Recognition*** và ***Algorithm Design***.





Trừu tượng hóa (Abstraction) và Tổng quát hóa (Generalization)

- Abstraction thường được thực hiện trước để đơn giản hóa và làm nổi bật bản chất của bài toán → giúp loại bỏ yếu tố gây nhiễu, giữ lại thông tin cốt lõi.
- Generalization thực hiện sau khi abstraction đã làm rõ cấu trúc dữ liệu và logic → giúp tìm ra quy luật hoặc mô hình áp dụng được cho nhiều tình huống.
- Nếu abstraction không đúng hoặc không đầy đủ → việc tổng quát hóa sẽ dựa trên thông tin sai lệch.
- Nếu abstraction tốt → mô hình hóa bài toán rõ ràng → dễ dàng phát hiện pattern và xây dựng giải pháp tổng quát.

Trừu tượng hóa (Abstraction) và Tổng quát hóa (Generalization)

Bài toán cụ thể: điểm danh sinh viên bằng nhận diện khuôn mặt tại một lớp học.

Abstraction:

- Loại bỏ chi tiết không cần thiết như màu tóc, biểu cảm, ánh sáng.
- Chuyển ảnh khuôn mặt thành vector đặc trưng → biểu diễn bản chất khuôn mặt.

Generalization:

- Mô hình hoạt động với sinh viên mới, lớp học khác, điều kiện khác (góc chụp, thiết bị).
- Giải pháp mở rộng dùng cho toàn bộ trường, hoặc áp dụng cho điểm danh nhân viên, khách ra vào.



Hình ảnh minh họa tạo bởi ChatGPT

Trừu tượng hóa (Abstraction) và Tổng quát hóa (Generalization)

Bài toán cụ thể:

- Quán ăn sinh viên thường xuyên ùn tắc vào giờ trưa.
- Thời gian chờ lâu, gây khó chịu và lãng phí thời gian học.

Abstraction:

- Loại bỏ thông tin như loại món ăn, tên khách, vị trí bàn. Giữ lại các yếu tố cốt lõi: số lượng người, thời gian xử lý mỗi lượt, cách xếp hàng.

Generalization:

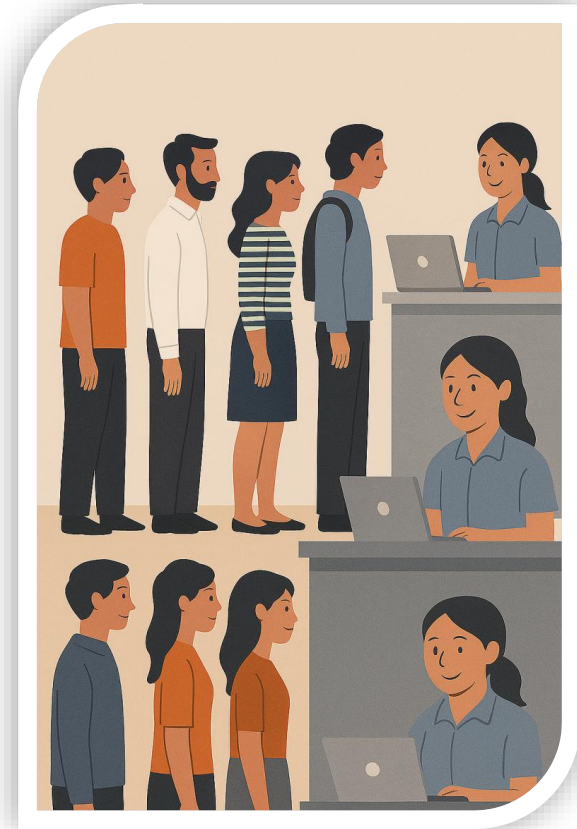
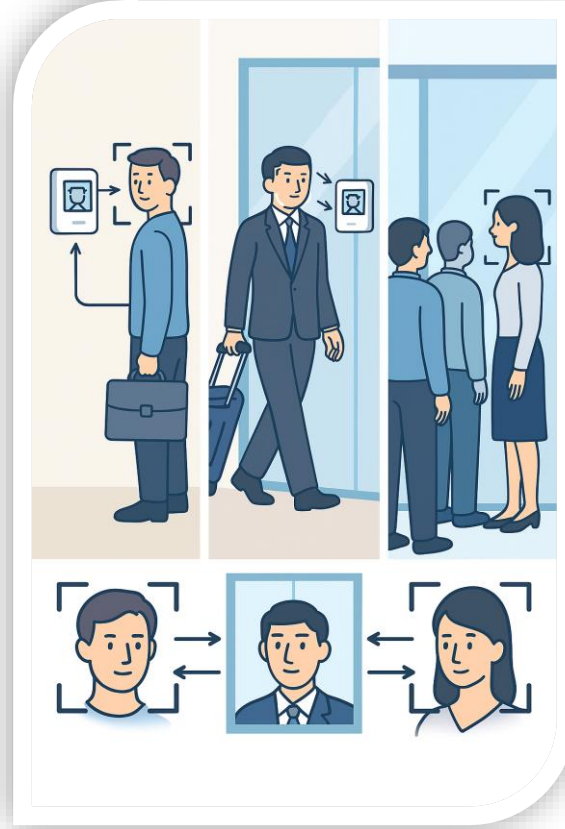
- Rút ra mô hình tổng quát: “bài toán điều phối hàng đợi”.
- Giải pháp mở rộng dùng cho ngân hàng, cửa hàng, trung tâm dịch vụ.



Hình ảnh minh họa tạo bởi ChatGPT



Trừu tượng hóa (Abstraction) và Tổng quát hóa (Generalization)



“Trừu tượng hóa để hiểu đúng, tổng quát hóa để giải rộng”



CHÂN THÀNH CẢM ƠN